

# 基于 Three.js 与 Mujoco WASM 的无人机安全走廊仿真

朱华天 Grok 4.6 GPT 5.6 Sol

2026 年 8 月 18 日

项目主页: <https://github.com/ZHT1235456/MujocoWASM>

## 摘要

本文记录一项面向浏览器的四旋翼安全走廊仿真实验：在获悉 Google DeepMind 已将 MuJoCo 物理引擎编译为 WebAssembly 之后，作者尝试把它与 Three.js 的实时三维渲染结合起来，做成可在网页中交互运行、也可经 Tauri 2 打包为桌面程序的仿真软件。问题定位来自近期阅读的 IEEE TAC 论文 [7]：在杂乱障碍环境中为四旋翼构造计及跟踪误差的安全超矩形管，再在管内用 Bézier 曲线与线性规划生成名义轨迹，并以几何跟踪控制驱动机体沿管飞行。前端由 Grok 4.6 开发，GPT 5.6 Sol 负责缺陷修复，Composer 2.5 完成 Tauri 2 桌面封装。本文依次介绍 Three.js 与 MuJoCo WASM 的官方定位、安全走廊方法、代码目录、模型协作分工，并给出在本地网页中实际操作后截取的界面。

## 1 引言

四旋翼无人机的到达-避障 (reach-avoid) 任务，要求机体在有限时间内进入目标区域，同时始终停留在工作域内并避开障碍。经典做法是先用采样规划得到航点，再用多项式拼接名义轨迹，最后依靠微分平坦性设计跟踪控制器。这一规划-跟踪范式工程上成熟，但若初始状态与名义轨迹不完全重合，通常并不能给出形式化的安全保证：跟踪误差一旦被障碍或推力饱和放大，机体就可能离开预先声明的安全集合。

Serry、Yuan、Chang 与 Liu 在 *IEEE Transactions on Automatic Control* 上的工作 [7] 针对这一问题提出了一条可计算的路径：先对几何跟踪控制做闭环稳定性再分析，得到与具体轨迹弱相关、可写成闭式的时变跟踪误差界；再把该误差界嵌入采样规划，用超矩形安全集构造一条“安全管” (safe tube / corridor)；最后在管内用分段 Bézier 曲线与启发式线性规划合成名义轨迹。这样，只要初始状态落在相应的相对内点集合内、且轨迹满足若干易检验的条件，位置、速度与推力约束便具有形式化保证。

笔者在了解到 MuJoCo 已提供官方 WASM 绑定 [4] 之后，产生了一个很直接的想法：物理步进可以在浏览器里完成，视觉层则交给 Three.js 去画更接近产品原型的场景。

于是本仓库把论文中的规划-跟踪流水线接到网页前端，四旋翼外观沿用 `reference/` 中按三视图标定的机体模型，动力学则由 `@mujoco/mujoco` 在 WASM 中积分。本文把“安全走廊”做成可看见、可调节、可打包的仿真器，对论文中的定理证明只作工程化近似。

后文安排如下。第 2 节与第 3 节分别依据官方文档介绍 Three.js 与 MuJoCo WASM；第 4 节概述安全走廊方法；第 5 节说明仿真实现、模型协作分工与代码目录；第 6 节给出实际操作截图；第 7 节简述桌面打包与软件体积；最后给出结语与参考文献。

## 2 Three.js：把三维内容放到网页上

Three.js 是一套开源的 JavaScript 三维库，官方站点与手册见 <https://threejs.org/> 与 [https://threejs.org/manual/\[1, 9\]](https://threejs.org/manual/[1, 9])。其 GitHub 仓库开宗明义地写道：项目目标是提供易用、轻量、跨浏览器、通用的三维库；当前主构建包含 WebGL 与 WebGPU 渲染器，SVG 与 CSS3D 渲染器则以插件形式提供。

官方手册 *Fundamentals* 指出，Three.js 与 WebGL 处于不同抽象层。WebGL 只绘制点、线与三角形，写出可用的三维应用通常需要大量样板代码；Three.js 把场景、灯光、阴影、材质、纹理以及三维数学封装起来。一个最小应用的骨架可以概括为：

1. 用 `WebGLRenderer`（或 `WebGPURenderer`）把场景画到 `<canvas>`；
2. 用 `Scene` 作为场景图的根；
3. 用相机（本项目采用 `PerspectiveCamera`）定义视锥；
4. 用 `Mesh` 把几何（`Geometry`）与材质（`Material`）组合起来，并挂到场景图中。

手册强调：`Renderer` 是 Three.js 中最核心的对象——把场景与相机交给它，它就把落在视锥内的部分画成二维图像。场景图是树状结构，子物体的位姿相对父物体定义；例如把四副旋翼挂在机身根节点下，移动根节点时桨叶会一起走。自 r147 起，官方推荐以 ES 模块方式引入：

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
```

本仿真正是按这一路径组织视觉层的。文件 `src/vis/scene.js` 创建带软阴影的 `WebGLRenderer`、ACES 色调映射、半球光与方向光，并用 `OrbitControls` 提供左键旋转、滚轮缩放、右键平移。四旋翼网格来自目录 `reference/drone` 中按实机三视图标定的机身、折叠臂、桨叶与云台。走廊、中心线、RRT 树是另一组半透明网格，由规划结果驱动，与物理引擎解耦。“物理在 WASM、外观在 Three.js”这一切分，使浏览器里的科研原型也能拥有接近产品演示的光照与材质。

## 3 MuJoCo 与官方 WASM 绑定

MuJoCo 的全称是 **M**ulti-**J**oint dynamics with **C**ontact，即“带接触的多关节动力学”。官方概述 [3] 将其定位为通用物理引擎，服务于机器人、生物力学、图形与动画、机

器学习等需要快速且准确地模拟铰接结构与环境交互的领域。该引擎最初由 Roboti LLC 开发，2021 年 10 月由 Google DeepMind 收购并免费开放，2022 年 5 月开源。运行时是面向性能调优的 C/C++ 库：模型用可读的 MJCF XML（亦可载入 URDF）描述，由内置解析器与编译器预分配底层数据结构，再在这些结构上做步进。官方文档明确指出，它既可用于控制综合、状态估计、系统辨识、逆动力学分析与机器学习并行采样，也可作为传统意义上的交互式仿真器。

对本项目更关键的是 2026 年前后成熟的官方 JavaScript/TypeScript 绑定 [4]。DeepMind 在仓库 `wasm/` 目录中声明：这是 MuJoCo 的权威 JS/TS 绑定，把核心引擎编译为高性能 WebAssembly 模块，并提供高层 API；绑定与主线同步演进，可通过 npm 安装 `@mujoco/mujoco`。该包是 ESM 模块，内含预编译的 `.wasm`、JavaScript glue 与 TypeScript 声明。文档特别提醒：打包器或开发服务器必须在运行时提供 `.wasm` 资源。

本仓库的接入方式与官方建议一致。`vite.config.js` 将 WASM 列为静态资源，并把 `@mujoco/mujoco` 排除出依赖预构建，以免 Vite 改写其定位逻辑；开发与预览服务器同时设置

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
```

以满足 `SharedArrayBuffer` 与 WASM 线程相关的跨源隔离要求。`src/sim/mujoco.js` 用

```
import loadMujoco from '@mujoco/mujoco';
import wasmUrl from '@mujoco/mujoco/mujoco.wasm?url';
```

加载引擎，再以 `MjModel.from_xml_string`（或回退到虚拟文件系统上的 `loadFromXML`）从运行时生成的 MJCF 构造 `MjModel/MjData`，每帧按渲染步长循环调用 `mj_step`。四旋翼以 `freejoint` 置于世界中，四个电机是作用在旋翼站点上的 `motor` 执行器：推力沿机体  $z$ ，偏航力矩由齿轮系数给出。障碍物是与视觉场景共用的轴对齐盒子。浏览器里看到的机体运动，由 WASM 中的刚体动力学积分驱动。

MuJoCo 的原生运行时是 C/C++ 库，官方 Python 绑定也已长期用于论文复现与批处理实验。若走 Python 路线，物理步进与可视化通常落在本地解释器、pip 环境以及 MuJoCo 自带查看器（或 Matplotlib）上，分享时对方需要装同一套依赖；若直接调用 C++ API，则需本机编译工具链，交付物是各平台各自的本地可执行文件。本项目希望同一份前端既能在浏览器里打开，也能经 Tauri 打成桌面程序，因此选用官方 WASM 绑定：核心引擎仍是那份 C/C++ 实现，经 Emscripten 编译后由 JavaScript 调用，运行环境收束为一个现代浏览器或系统 WebView。

## 4 安全走廊：从形式化保证到可交互仿真

### 4.1 问题陈述

沿用文献 [7] 的记号。设工作域  $\mathcal{X}_o$  与障碍  $\mathcal{X}_u$  均为超矩形（或超矩形的并），目标  $\mathcal{X}_t \subseteq \mathcal{X}_o \setminus \mathcal{X}_u$ 。四旋翼在忽略气动阻力时的刚体方程为

$$\dot{p} = v, \quad \dot{v} = -ge_3 + m^{-1}f Re_3, \quad (1)$$

$$\dot{R} = R\hat{\omega}, \quad \dot{\omega} = J^{-1}(-\omega \times J\omega + \tau). \quad (2)$$

到达-避障任务要求存在有限时间  $T$ ，使位置始终位于  $\mathcal{X}_o \setminus \mathcal{X}_u$ ，速度与推力满足给定盒约束，且  $p(T) \in \mathcal{X}_t$ 。论文强调：保证须覆盖一个含名义初值于其相对内部的初始集合，即名义轨迹附近的一整族解都要满足安全约束。

### 4.2 几何跟踪与时变误差界

跟踪层采用  $SE(3)$  上的几何控制 [6, 7]。定义位置、速度、姿态与角速度误差  $e_p, e_v, e_R, e_\omega$ ，推力与力矩取

$$f = F_d \cdot Re_3, \quad F_d = -k_p e_p - k_v e_v + mge_3 + m\ddot{p}_d, \quad (3)$$

$$\begin{aligned} \tau = & -k_R e_R - k_\omega e_\omega + \omega \times J\omega \\ & - J(\hat{\omega} R^\top R_d \omega_d - R^\top R_d \dot{\omega}_d). \end{aligned} \quad (4)$$

文献 [7] 的一个关键结论是：对任意正增益，跟踪误差动力学具有局部指数稳定性；并且可以导出与具体轨迹无关（只要轨迹满足若干易加约束）的闭式时变误差界。该界被用来收缩工作域、膨胀障碍，从而把“跟踪会偏多少”预先扣进几何规划里。

本仓库在几何控制模块中实现推力-力矩律，再由混控把总推力与三轴力矩分配到四个电机。仿真里用指数衰减裕度

$$\ell(t) = \ell_0 \exp(-\lambda t)$$

近似论文中的时变界，作为超矩形安全集计算的垫层。论文中的 Lyapunov 估计仍以原文为准，浏览器侧只保留这一便于计算的指数形式。

### 4.3 超矩形安全管与管内 Bézier-LP

规划层对应论文第 VII 节的管构造与轨迹合成，实现上分为三步。

1. **采样扩展**。安全管规划模块在经  $\ell(t)$  与机身半径垫层后的自由空间中做 RRT 式扩展 [5]。每个新节点不仅记录位置与到达时间，还调用超矩形例程 `safeRect`，计算仍避开膨胀障碍、且包含该点的轴对齐安全超矩形。

2. **安全管**。沿从起点连到终点的节点链，得到一串随时间变化的超矩形，即安全走廊。可视化时画成半透明青绿色盒子，中心线画成黄色折线。
3. **管内多项式**。`src/plan/bezier-lp.js` 用 `glpk.js`[2] 求解论文 Algorithm 2 风格的线性规划：决策变量是各段 Bézier 控制点，约束包括段间  $C^2$  连续、控制点落在对应超矩形内、以及速度/加速度盒约束。若当前时长不可行，则按比例拉长时间重试，直到得到可行轨迹或达到尝试上限。

起飞后，`src/main.js` 按仿真时间在分段 Bézier 上取值，把位置、速度、加速度送给几何控制器；MuJoCo 以 2ms 步长、RK4 积分推进刚体。HUD 用若干帧确认机体是否仍在当前盒子内，避免单帧数值抖动造成误报。

## 5 系统设计与模型分工

### 5.1 总体架构

系统按“场景描述 – 规划 – 控制 – 物理 – 可视化 – 交互”六段划分，全部运行在浏览器主线程与 WASM 中，无需后端服务。数据流可以写成：

起终点与采样参数 → RRT 超矩形管 → Bézier-LP 轨迹 → 几何控制 + 混控 → MuJoCo  
mj\_step → Three.js 同步位姿

坐标约定需要单独说明。Three.js 默认  $Y$  向上，MuJoCo 默认  $Z$  向上。`src/coords.js` 给出

$$(x, y, z)_{\text{three}} = (x, z, -y)_{\text{mj}},$$

四元数亦做相应置换，避免视觉与物理各说各话。

### 5.2 四旋翼外观模型

视觉机体来自仓库 `reference/drone`。该模块最初是独立的 Three.js 建模练习：以机身全长 183mm 为标定基准，从三视图像素量测电机间距、桨盘直径、前后臂后掠角与云台尺寸，再拆成 `fuselage.js`、`arm.js`、`propeller.js`、`gimbal.js` 装配。机体坐标系取 + $X$  右、+ $Y$  上、+ $Z$  机头，原点在机身包络几何中心。Vite 把 `@drone` 别名指向该目录，仿真场景直接 `createDrone()`，并按电机转速旋转桨叶。物理侧用半径 0.28m 的球体作为碰撞包络，以减轻 MJCF 负担；外形三角面只参与显示。电机站点坐标与视觉模型中的电机轴心对齐，保证混控力臂与画面一致。

### 5.3 模型分工

开发按模型能力拆开，仓库提交也大致沿“实现 → 修复 → 交互打磨 → 桌面打包”排列：

- **Grok 4.6:** 前端主体——页面结构、Three.js 场景、规划可视化、面板绑定，以及把论文算法改写成可在浏览器中运行的 JavaScript 模块。

- **GPT 5.6 Sol:** 缺陷修复。实际过程中出现过电机分配符号错误、Bézier-LP 过早报不可行、桨叶被深度测试裁掉、走廊违约闪烁等问题，均在后续提交中逐项修正。
- **Composer 2.5:** 在前端可运行之后，用 Tauri 2 把同一套 Vite 应用封装为 Windows 桌面程序（NSIS 安装包），并配置跨源隔离响应头，使 WASM 在 WebView2 中与浏览器行为一致。

## 5.4 代码目录结构

仓库根目录的主要条目如下。

表 1: 仓库顶层与 `src/` 目录说明

| 路径                              | 说明                                                                                                     |
|---------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>index.html</code>         | 单页入口：视口、HUD、任务/显示/规划侧栏                                                                                 |
| <code>package.json</code>       | 依赖 <code>three</code> 、 <code>@mujoco/mujoco</code> 、 <code>glpk.js</code> ；脚本含 <code>dev/tauri</code> |
| <code>vite.config.js</code>     | WASM 资源、COOP/COEP、 <code>@drone</code> 别名、Tauri 开发端口                                                   |
| <code>src/main.js</code>        | 应用主循环：规划、起飞、跟拍、违约检测                                                                                    |
| <code>src/world.js</code>       | 由场景常数生成 MJCF 字符串                                                                                       |
| <code>src/world-scene.js</code> | 工作域、起终点、障碍盒、质量与电机站点                                                                                    |
| <code>src/coords.js</code>      | MuJoCo Z-up 与 Three.js Y-up 互转                                                                         |
| <code>src/sim/mujoco.js</code>  | WASM 加载、建模、 <code>mj_step</code>                                                                       |
| <code>src/plan/</code>          | 超矩形、RRT 管、Bézier、碰撞与 LP                                                                                |
| <code>src/control/</code>       | 几何控制、混控与悬停推力                                                                                           |
| <code>src/vis/</code>           | 渲染器、场景、机体同步、走廊网格                                                                                       |
| <code>src/ui/panel.js</code>    | 侧栏绑定、忙碌遮罩、指标刷新                                                                                         |
| <code>reference/drone/</code>   | 按三视图标定的四旋翼外观                                                                                           |
| <code>paper/</code>             | TAC 论文 Markdown 文本，供对照实现                                                                               |
| <code>scripts/</code>           | 几何控制、管、LP、MuJoCo 与飞行的节点测试                                                                              |
| <code>src-tauri/</code>         | Tauri 2 工程：窗口、图标、NSIS 打包                                                                               |
| <code>docs/</code>              | 本文 L <sup>A</sup> T <sub>E</sub> X 源文件与界面截图                                                            |

`src/plan` 内部再按论文对象拆分：`hyperrect.js` 做轴对齐盒的膨胀/收缩与安全矩形；`rrt-tube.js` 产出带时间戳的盒子链；`bezier.js/bezier-tube.js` 负责 Bernstein 基与管内采样；`bezier-lp.js` 调用 GLPK。测试脚本不依赖浏览器，便于在改 LP 约束或混控符号后快速回归。

## 6 界面与运行实录

本地以 Vite 开发服务器打开页面（`npm run dev`）。默认俯视工作域：浅色障碍块散布在深色地面上，四旋翼停在左下角起点，右侧是任务、显示、规划与起终点面板。加载 WASM 完成后，状态栏提示“就绪：点击规划路径”，走廊状态为“待机”，如图 1。

点击“规划路径”后，界面进入忙碌遮罩，主线程进行 RRT 扩展与线性规划。一次典型运行得到路径长度 38.29 m。规划成功后，青绿色超矩形管从起点绕过门柱、矮墙与塔楼连到右上角终点，黄色中心线穿管而过，走廊状态变为“已规划”，如图 2。打开“RRT 树”后，还能看见采样阶段留下的浅色树枝，用以对照最终被保留的那条盒子链，如图 3。

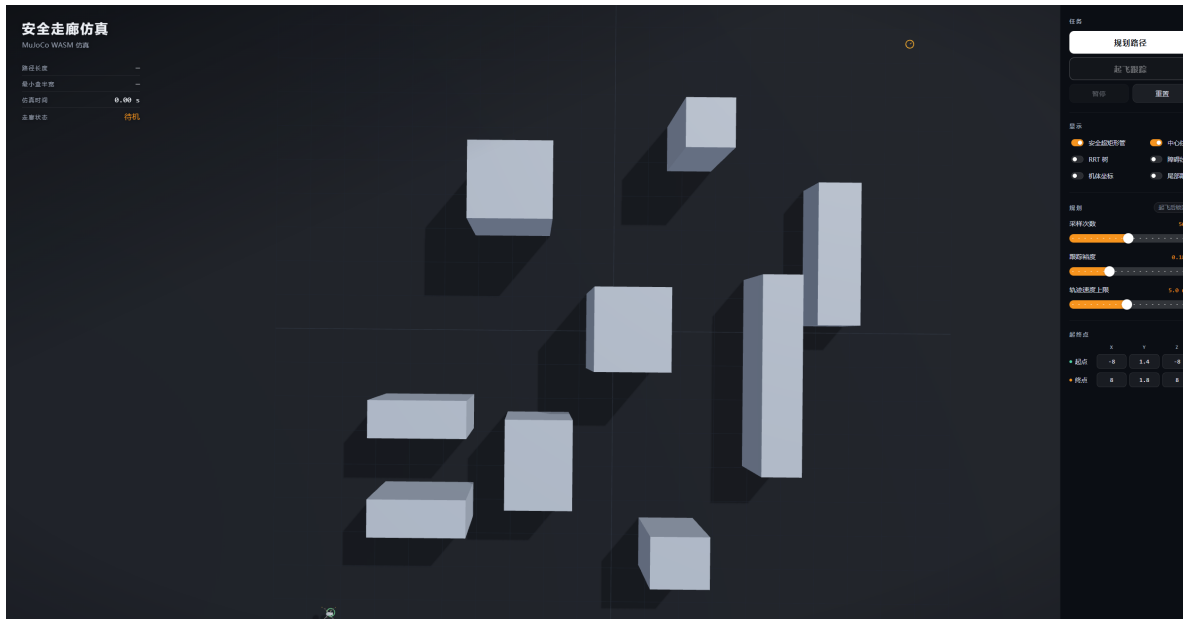


图 1: 仿真启动后的俯视场景。左侧 HUD 给出路径长度、最小盒半宽、仿真时间与走廊状态；右侧可调节采样次数、跟踪裕度与速度上限。



图 2: 规划完成后的安全超矩形管与中心线。起飞按钮此时才可用，避免在无名义轨迹时启动几何跟踪。



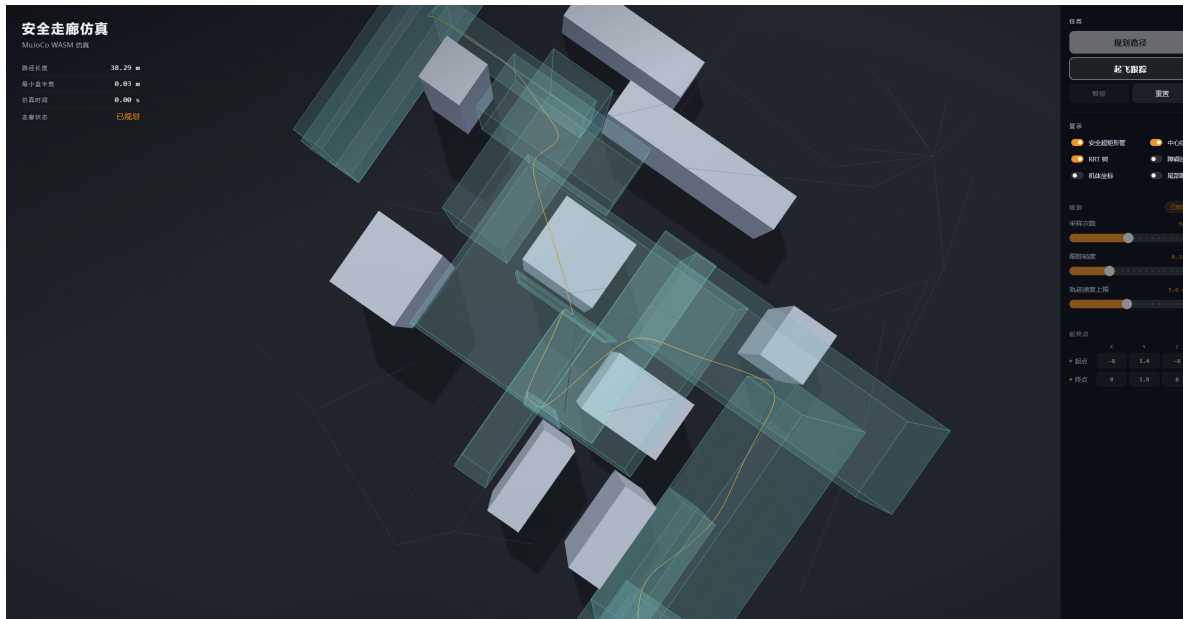


图 3: 打开 RRT 树显示后的同一规划结果。树展示了采样搜索的覆盖范围，走廊则是从树中提取并经安全矩形计算得到的可行通道。

点击“起飞跟踪”后，几何控制开始把 MuJoCo 中的自由刚体拉向 Bézier 参考。图 4 截取自飞行过程：仿真时间约 17s，走廊状态为“管内”，状态栏显示“几何控制跟踪安全管”。再打开“尾部跟拍”，相机落到机体后方，可以看清 reference/drone 的机身、桨叶与管内黄色参考线。机体到达终点后改为定点悬停，图 5 即悬停时刻的跟拍画面，仿真时间约 92s，状态为“到达终点，定点悬停”。

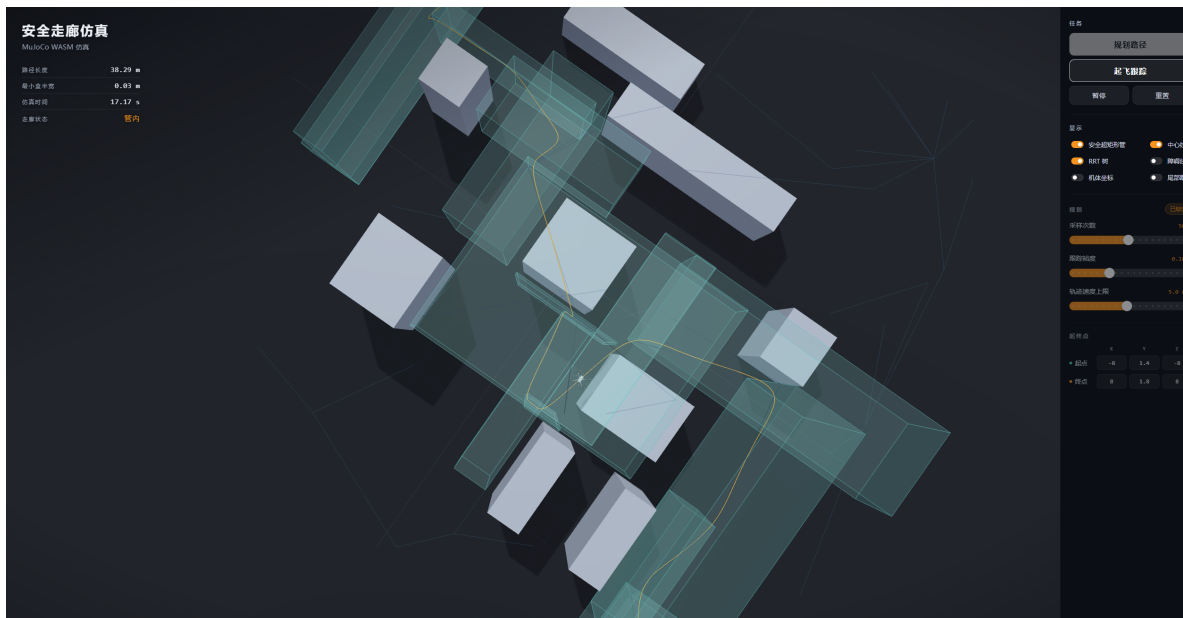


图 4: 几何控制跟踪过程中的俯视。机体沿黄色中心线穿过走廊，HUD 指示仍在管内。

上述操作并未改动默认起终点（起点  $(-8, 1.4, -8)$ ，终点  $(8, 1.8, 8)$ ）与默认采样次数 5000。交互上还有两点值得记下：规划与飞行期间会锁定起终点和采样滑条，以免中途改参数造成管与控制器不一致；走廊违约需连续若干帧确认后才改写 HUD，以免数值抖动造成误报。



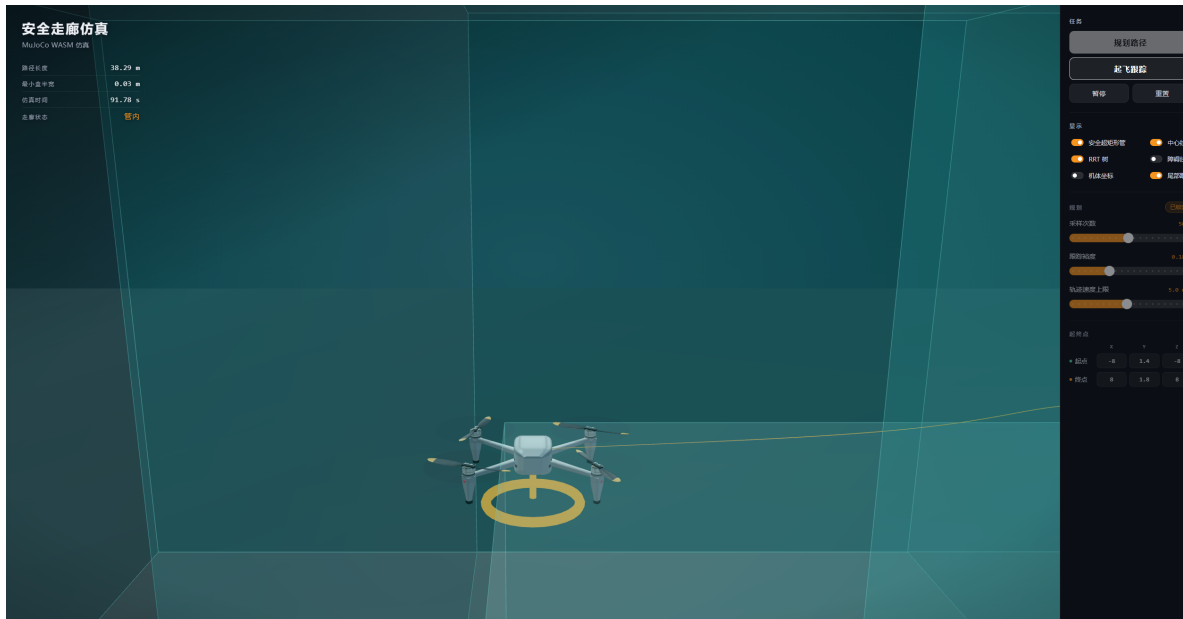


图 5: 尾部跟拍视角下到达终点并悬停。半透明走廊盒、中心线与按三视图建模的四旋翼同时可见。

## 7 用 Tauri 2 打包桌面程序

网页可以满足演示，科研现场往往还希望“双击即开”。Tauri 2 的官方说明 [8] 将其定位为：用系统自带的 WebView 构建体积小、启动快的桌面与移动应用，前端可以是任何能编译为 HTML/CSS/JS 的技术，后端按需使用 Rust。Windows 上它使用 WebView2，安装包主要包含应用自身的前端资源与 Rust 壳程序，WebView 由操作系统提供。

`src-tauri/` 中的配置把产品名设为“安全走廊仿真”，开发期把窗口指到 Vite 的 `http://localhost:5173`，发布期加载 `dist/`。为了让 WebView 与浏览器一样能加载 MuJoCo WASM，`tauri.conf.json` 同样下发 COOP/COEP 响应头。打包目标当前为 Windows NSIS。相关 npm 脚本为 `npm run tauri:dev` 与 `npm run tauri:build`。这一层由 Composer 2.5 完成，前端代码无需为桌面单独分叉。

体积也是这一技术栈的直接结果。Vite 生产构建后的 `dist/` 约 10.6 MB，其中 `mujoco.wasm` 约 9.65 MB，`Three.js` 与业务脚本合计约 0.9 MB。本机实测 `drone-corridor.exe` 约 4.7 MB，NSIS 安装包经 LZMA 压缩后约 3.3 MB。官方文档称最小 Tauri 应用可以小于 600 KB[8]；本软件的体量几乎全部来自那份 WASM 物理引擎。常见 Electron 应用会随安装包携带完整 Chromium，体量往往达到数十至上百 MB；Python 方案若计入解释器、NumPy 与 MuJoCo 本体，部署目录通常也明显更重。WASM 加系统 WebView，把能跑 MuJoCo 的四旋翼演示收成一个可随仓库分发的小安装包。

## 8 结语

本文把三件事情接到同一份可运行的前端上：MuJoCo 官方 WASM 让物理引擎出现在网页里；`Three.js` 把科研场景做成可演示的三维界面；IEEE TAC 的安全走廊工作 [7] 为“在障碍中飞过去”提供了计及跟踪误差的安全语义。模型分工同样清楚：Grok 4.6 铺

开前端与算法模块，GPT 5.6 Sol 对着具体失效模式修改，Composer 2.5 把同一套页面送进 Tauri 窗口。

当前实现仍是论文方法的工程近似。时变误差界被写成指数裕度，超矩形与 LP 约束针对浏览器算力做了裁剪，碰撞包络取球体，外形网格仅用于显示。后续可以把文献中的闭式界与奇异回避条件更严格地搬进 `src/plan`，并用脚本批量统计越管率、跟踪误差包络是否被界罩住。那些工作仍适合按模型分工推进：把“安全”的定义写成能在 WASM 与 WebGL 里跑起来的代码。

源码与本文所用截图见：

<https://github.com/ZHT1235456/MujocoWASM>

## 参考文献

- [1] Ricardo Cabello and Three.js Contributors. three.js: JavaScript 3D library. <https://threejs.org/>, 2026. Official documentation: <https://threejs.org/docs/>; manual: <https://threejs.org/manual/>.
- [2] glpk.js Contributors. GLPK.js: GNU linear programming kit for JavaScript. <https://www.npmjs.com/package/glpk.js>, 2024.
- [3] Google DeepMind. MuJoCo: Multi-joint dynamics with contact. <https://mujoco.readthedocs.io/en/stable/overview.html>, 2026. Source repository: <https://github.com/google-deepmind/mujoco>.
- [4] Google DeepMind. MuJoCo JavaScript bindings (WASM). <https://github.com/google-deepmind/mujoco/tree/main/wasm>, 2026. npm package @mujoco/mujoco.
- [5] Steven M. LaValle. Rapidly-exploring random trees: A new tool for path planning. In *Technical Report 98-11, Computer Science Department, Iowa State University*, 1998.
- [6] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). *Proceedings of the 49th IEEE Conference on Decision and Control*, pages 5420–5425, 2010. doi: 10.1109/CDC.2010.5717652.
- [7] Mohamed Serry, Yating Yuan, Haocheng Chang, and Jun Liu. Reach-avoid control synthesis for a quadrotor UAV with formal safety guarantees. *IEEE Transactions on Automatic Control*, 71(8):4939–4955, August 2026. doi: 10.1109/TAC.2026.3664766.
- [8] Tauri Contributors. Tauri 2 documentation. <https://v2.tauri.app/start/>, 2026.
- [9] Three.js Authors. three.js manual: Fundamentals. <https://threejs.org/manual/en/fundamentals.html>, 2026.